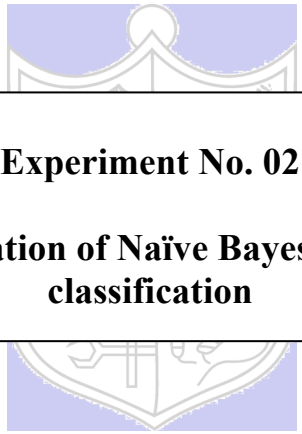


Experiment No. 02

Title: Implementation of Naïve Bayesian algorithm for classification

Batch: C3 **Roll No: 16014225801** **Experiment No: 02**

Aim: Implementation of Naïve Bayesian algorithm for classification

---

Resources Used: Python

Theory:

Bayesian Classifier

A Bayesian classifier is a simple probabilistic classifier. Bayesian classifier can predict membership probabilities such as the probabilities that a sample belongs to a particular class or groupings.

Bayesian classification is based on Bayes theorem and this technique tends to be highly accurate and fast, making it useful on large databases.

Naïve Bayesian Classification Algorithm is as follows:

1) Training Set

Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n-dimensional attribute vector,

$$X = (x_1, x_2, \ldots, x_n)$$

depicting n measurements made on the tuple from n attributes, respectively,

$$A_1, A_2, \ldots, A_n.$$

2) Class Prediction

Suppose that there are m classes,

$$C_1, C_2, \ldots\ldots, C_m$$

Given a tuple, X, the classifier will predict that X belongs to the class having the highest posterior probability, conditioned on X. That is, the Naïve Bayesian classifier predicts that tuple X belongs to the class C_i if and only if the class C_i for which $P(C_i \mid X)$ is maximized is called the *maximum posteriori hypothesis*.

3) Using Bayes' Theorem

As P(X) is constant for all classes, only $P(X/C_i)\ P(C_i)$ needs to be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is,

$$P(C_1) = P(C_2) = \ldots\ldots = P(C_m),$$

and we would therefore maximize $P(X/C_i)$. Otherwise, we maximize $P(X/C_i)\ P(C_i)$. Note that the class prior probabilities may be estimated by

$$P(C_i) = |C_i,D| / |D|$$

where

- $|C_i,D|$ is the number of training tuples of class C_i in D.
- |D| is the total number of training tuples.

This presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus, we can easily estimate the probabilities

$$P(x_1/C_i), P(x_2/C_i), \ldots\ldots, P(x_n/C_i)$$

from the training tuples. Recall that here x_k refers to the value of attribute A_k for tuple X. For each attribute, we look at whether the attribute is categorical or continuous-valued.

4) Maximum Probability (Max.)

Sample X is therefore assigned to class C_i if and only if

$$P(X/C_i) \cdot P(C_i) > P(X/C_j) \cdot P(C_j) \text{ For } 1 \leq j \leq m, j \neq i.$$

In other words, it is assigned to the class C_i for which $P(X/C_i) \cdot P(C_i)$ is maximum.

Procedure:

1. Identify attributes suitable for applying classification algorithm
2. Implement Naïve Bayesian on your dataset.
3. Apply Naïve Bayesian to classify unknown tuple.

Program & Output:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

df = pd.read_csv("bombay.csv")

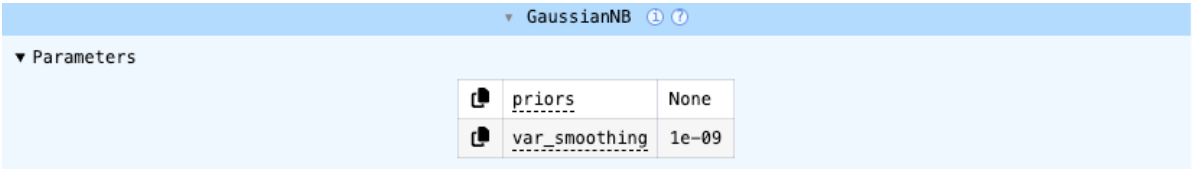
def classify(temp):
    if temp < 70:
        return "Cold"
    elif temp <= 80:
        return "Warm"
    else:
        return "Hot"

df["Temp_Class"] = df["Temp"].apply(classify)

X = df[["Month", "Day", "Year"]].values
y = df["Temp_Class"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Output:



▼ Fitted attributes

Name	Type	Value
class_count_	ndarray[float64](3,)	[59.,5531.,1822.]
class_prior_	ndarray[float64](3,)	[0.01,0.75,0.25]
classes_	ndarray[<U4](3,)	['Cold','Hot','Warm']
epsilon_	float64	7.726e-08
n_features_in_	int	3
theta_	ndarray[float64](3, 3)	[[4. , 15.47,2006.29], [6.87, 15.76,2007.6], [5.3 , 15.58,2005.84]]
var_	ndarray[float64](3, 3)	[[16.98,70.83,57.26], [8.54,77.37,53.22], [19.75,77.11,50.74]]

Input:

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy: 0.7637540453074434
```

Input:

```
sample = [[7,15,2002]]
prediction = model.predict(sample)
print(prediction)
```

Output:

```
['Hot']
```

Post Lab Questions Answer:

1. What are advantages and disadvantages of Bayesian Classification?

Bayesian Classification is a simple, fast, and efficient machine learning algorithm that performs well even on large datasets. It requires less training data, is easy to implement, and can handle both categorical and numerical data. However, it assumes that all features are independent of each other, which is often not true in real-world datasets. If the features are highly correlated or if a feature value is absent in the training data, the classifier's accuracy may decrease.

2. Comment on Laplacian correction.

Laplacian correction, also known as **Laplace smoothing**, is a technique used in Naïve Bayes to avoid zero probability problems. When a particular feature value does not appear in the training data for a class, its probability becomes zero, causing the overall prediction probability to become zero. Laplace smoothing adds a small value (usually 1) to each feature count, ensuring that every possible feature value has a non-zero probability. This makes the classifier more robust and improves its prediction accuracy, especially when dealing with unseen data.

Conclusion:

The Naïve Bayes classification algorithm was successfully implemented using Python & the model was trained, tested, and achieved an accuracy of 76.38%. It correctly classified an unknown sample, demonstrating that Naïve Bayes is a simple, efficient, and effective algorithm for classification tasks.

References:

Books/ Journals/ Websites:

1. Stephen Marsland, “Machine learning An Algorithmic Perspective CRC”, Press, 2015
2. Christopher M. Bishop “Pattern Recognition and Machine Learning Springer Science, Business Media 2006
3. Simon Haykin, “Neural Networks and Learning Machines” Pearson Education, 2009